

Modules and Packages in Python

Python Programming for IndiaAI Students

Presented by: Lovnish Verma

Project Engineer & Faculty | NIELIT Ropar, Punjab

Learning Objectives

By the end of this session, you will be able to:

1

Understand what modules and packages are

5

Create custom modules and packages

2

Explain the need for modular programming

6

Understand the role of `__init__.py`

3

Identify different types of modules

7

Apply Python best practices in real projects

4

Import and use modules effectively

8

Prepare for technical interviews on this topic

Why Do We Need Modules?

Problems in Large Programs

- ⚠ Giant, unmanageable codebase
- ⚠ Hard to maintain & debug
- ⚠ Poor readability
- ⚠ Code duplication everywhere
- ⚠ No clear separation of concerns

Modules!

With Modular Programming

- ✓ Small, focused files
- ✓ Easy to maintain & test
- ✓ Clean, readable structure
- ✓ Reuse code across projects
- ✓ Team collaboration made easy

What is a Module?

"A module is a Python file (.py) containing functions, classes, and variables that can be reused in other programs."

calculator.py

```
def add(a, b):  
    return a + b  
  
def subtract(a, b):  
    return a - b  
  
def multiply(a, b):  
    return a * b
```



Reusability

Write once, use anywhere



Maintainability

Easier to update and fix



Organization

Logical code grouping



Less Duplication

Avoid repetitive code

Types of Modules in Python

Built-in Modules

- math, os, random
- datetime, sys, shutil
- Part of Python

Python Modules

Third-Party Modules

- NumPy, Pandas,
- Sci-kit learn, PyTorch, Tensorflow
- Installed via pip

User-Defined Modules

- Created by developers
- calculator.py
- utils.py

Built-in Modules — Python Standard Library

Python ships with 200+ built-in modules. No installation needed — just import and use!



math

Mathematical functions
sqrt, sin, cos, pi, log



random

Random values
randint, choice, shuffle



os

OS interaction
path, getcwd, listdir



datetime

Date and time
now, date, timedelta



statistics

Stats calculations
mean, median, stdev

```
# Example: Using Built-in Modules
```

```
import math
import random
```

```
print(math.sqrt(25))      # Output: 5.0
print(random.randint(1, 10)) # Random number between 1-10
```

Popular Built-in Modules — Quick Reference

Module	Purpose	Example Usage
math	Mathematical functions (sqrt, log, trig)	<code>math.sqrt(16)</code> → 4.0
random	Generate random numbers and choices	<code>random.randint(1, 100)</code>
os	Interact with operating system	<code>os.getcwd()</code> , <code>os.listdir()</code>
datetime	Work with dates and times	<code>datetime.datetime.now()</code>
statistics	Statistical calculations	<code>statistics.mean([1,2,3])</code>
sys	Python interpreter info and control	<code>sys.version</code> , <code>sys.exit()</code>
json	Parse and write JSON data	<code>json.loads('{"key": 1}')</code>
re	Regular expressions for text	<code>re.findall(r'\d+', text)</code>

User-Defined Modules

"Modules created by developers for their own specific needs — saved as .py files and reused across projects."

1

Create file

Save as calculator.py

2

Write functions

Define reusable functions

3

Save the file

In your project folder

4

Import it

Use in other scripts

calculator.py — Our custom module

```
# calculator.py - A simple user-defined module
def add(a, b): return a + b
def subtract(a, b): return a - b
def multiply(a, b): return a * b
```

Third-Party Modules & pip

Thousands of powerful libraries built by the Python community — installed via pip package manager.

```
$ pip install pandas          # Install pandas library
$ pip install numpy matplotlib scikit-learn # Install multiple at once
```

NumPy

Array and matrix computation for scientific computing

Pandas

Data manipulation and analysis with DataFrames

Matplotlib

2D plotting and data visualization library

TensorFlow

Machine learning and deep neural network framework

Requests

HTTP requests for web APIs and web scraping

Flask

Lightweight web application framework

Importing Modules — How it Works

Write import
statement

Python searches
for module

Module is
loaded into memory

Functions are
now available

```
# Step 1: Import the module
```

```
import math
```

```
# Step 2: Use the module's functions (module.function syntax)
```

```
result = math.sqrt(16)
```

```
print(result)          # Output: 4.0
```

```
area = math.pi * (5 ** 2)
```

```
print(area)            # Output: 78.53981...
```

Different Ways to Import Modules

`import math`

Recommended

```
import math
print(math.sqrt(16))
```

- ✓ Clear namespace
- ✓ Avoids conflicts

✗ Longer to type

`from math import sqrt`

Common

```
from math import sqrt
print(sqrt(16))
```

- ✓ Direct access
- ✓ Shorter syntax

✗ May clash with local names

`from math import *`

Avoid

```
from math import *
print(sqrt(16))
```

- ✓ Import everything
- ✓ Quick to type

✗ Pollutes namespace
✗ Hard to debug

`import math as m`

Useful

```
import math as m
print(m.sqrt(16))
```

- ✓ Custom shorthand
- ✓ Clean code

✗ Less obvious for beginners

Module Search Path — How Python Finds Modules

When you write 'import module_name', Python searches these locations in order:

1

Current Directory

The folder where your script is running

2

PYTHONPATH Directories

Directories in the PYTHONPATH environment variable

3

Standard Library

Python's built-in modules (e.g., math, os, json)

4

Site-Packages

Third-party packages installed via pip

Check sys.path:

```
import sys

for path in sys.path:
    print(path)
```

If module not found
→ `ModuleNotFoundError`

Creating a Custom Module — Step by Step

STEP 1: Open your code editor
(VS Code, IDLE, etc.)

STEP 2: Write your Python
functions and variables

STEP 3: Save the file as
calculator.py

STEP 4: Place it in your project
folder

Project Structure

 my_project/

 calculator.py ← Our module

 main.py ← Uses module

 tests.py ← Tests

calculator.py

```
"""A simple calculator module"""

def
add(a, b): return a + b
def subtract(a, b): return a - b
def multiply(a, b): return a * b
def divide(a, b): return a / b if b != 0 else None
```

Pro Tips

- Use lowercase names: calculator.py ✓ Calculator.PY ✗
- Avoid naming files after built-in modules (e.g. math.py)
- Add docstrings to document your module's purpose

Using a Custom Module

calculator.py
(Module file)



import
calculator



calculator.add
(5, 3)

Output: 8

main.py — Using our calculator module

```
import calculator

# Call functions using module.function() syntax
result1 = calculator.add(5, 3)
result2 = calculator.subtract(10, 4)
result3 = calculator.multiply(3, 7)

print("Add 5+3 =", result1) # Add 5+3 = 8
print("Sub 10-4 =", result2) # Sub 10-4 = 6
print("Mul 3x7 =", result3) # Mul 3x7 = 21
```

Practice — Build a Student Utility Module

Let's build a real-world module together!

`calculate_marks()`

Sum marks across subjects

`calculate_percentage()`

Get % from total marks

`assign_grade()`

Return A/B/C/D/F grade

 `student_utils.py`

```
def calculate_marks(subjects_list):  
    return sum(subjects_list)  
  
def calculate_percentage(total, max_marks=500):  
    return (total / max_marks) * 100  
  
def assign_grade(percentage):  
    if percentage >= 90: return "A"  
    elif percentage >= 75: return "B"  
    elif percentage >= 60: return "C"
```

What is a Package?

"A package is a collection of related modules organized in a directory (folder), allowing hierarchical structuring of the Python module namespace."

MODULE

Single .py file

```
calculator.py  
_____  
    add()  
    subtract()  
    multiply()
```



Scale up!

PACKAGE

Folder of related modules

```
student_package/  
  
__init__.py  
  
marks.py  
  
attendance.py  
  
reports.py
```

Package Structure — Organizing Your Code

Folder Structure

```
student_package/  
├── __init__.py  
│   └── Required marker file  
├── marks.py  
│   └── calculate_marks()  
├── attendance.py  
│   └── mark_present()  
└── reports.py  
    └── generate_report()
```

`marks.py`

```
def calculate_marks(marks):  
    return sum(marks)  
  
def average(marks):  
    return sum(marks)/len(marks)
```

`attendance.py`

```
def mark_present(name):  
    print(f'{name} is present')  
  
def get_percentage(p, t):  
    return (p/t)*100
```

`reports.py`

```
def generate_report(name,  
                    marks, att):  
    print(f'Report: {name}')  
    print(f'Marks: {marks}')
```

Creating a Package — Step by Step

1

Create the Package Folder

Create a directory named `student_package`

```
mkdir student_package
```

2

Add `__init__.py`

Create empty or with package-level code

```
touch student_package/__init__.py
```

3

Add Module Files

Create `marks.py`, `attendance.py`, etc.

```
touch student_package/marks.py
```

4

Write Module Code

Add functions to each module file

```
# (write functions in each .py file)
```

5

Import & Use

Import from your package in `main.py`

```
from student_package import marks
```

Importing from Packages

Method 1: Full path import

```
import student_package.marks
```

```
student_package.marks.calculate_marks([85, 90])
```

💡 *Most explicit — clear where function comes from*

Method 2: Import module from package

```
from student_package import marks
```

```
marks.calculate_marks([85, 90])
```

💡 *Common pattern — shorter but still clear*

Method 3: Import specific function

```
from student_package.marks import  
calculate_marks
```

```
calculate_marks([85, 90])
```

💡 *Shortest syntax — best for frequently used functions*

Method 4: Alias import

```
from student_package import marks as m
```

```
m.calculate_marks([85, 90])
```

💡 *Useful when module name is long or conflicts*

Understanding `__init__.py`

`__init__.py` is the initialization file that tells Python: "This directory is a package, not just a folder."



Marks as Package

A directory with `__init__.py` is recognized as a Python package by the interpreter



Initialization Code

Code here runs automatically when the package is first imported



Control Exports

Use `__all__` to define what gets exported with 'from package import *'



Simplify Imports

Pre-import modules inside `__init__.py` to create shortcuts for users

```
# student_package/__init__.py
__all__ = ["marks", "attendance", "reports"]
```

```
# Controls what is exported
```

Real-World Python Packages — Industry Ecosystem

These packages are built from modules and packages — exactly what you're learning today!

NumPy

★★★★★

Scientific Computing

Modules: core, linalg, fft, random, polynomial

Pandas

★★★★★

Data Analysis

Modules: core, io, plotting, testing, tseries

Matplotlib

★★★★☆

Data Visualization

Modules: pyplot, axes, figure, animation, patches

Scikit-Learn

★★★★★

Machine Learning

Modules: linear_model, tree, svm, metrics, preprocessing

TensorFlow

★★★★★

Deep Learning

Modules: keras, data, lite, distribute, hub

Flask

★★★★☆

Web Development

Modules: routing, templating, sessions, blueprints

Module vs Package — Side by Side Comparison

Feature	Module	Package
Definition	Single .py file with code	Directory containing modules
Physical form	calculator.py	student_package/ folder
Required file	—	__init__.py
Complexity	Small to Medium	Medium to Large
Import syntax	import calculator	import student_package
Reusability	Within a project	Across multiple projects
Organization	Single concern	Multiple related concerns
Example	math, json, os	numpy, pandas, tensorflow

Common Import Errors & How to Fix Them

⚠ ModuleNotFoundError

Cause: Python can't find the module

```
import nonexistent_module
```

✓ **Fix:** Check module name spelling, ensure it's installed (pip install), and verify you're in the right directory

⚠ ImportError

Cause: Module found but specific item not found

```
from math import nonexistent
```

✓ **Fix:** Check the function/class name. Use `dir(module)` to see available items

⚠ CircularImportError

Cause: Module A imports B, and B imports A

```
a.py imports b; b.py imports a
```

✓ **Fix:** Restructure code to remove circular dependency or import inside function

⚠ NameError (wrong filename)

Cause: File named `math.py` shadows built-in

```
Created math.py in project folder
```

✓ **Fix:** Rename your file! Never name files after built-in modules

Best Practices for Modules & Packages



Meaningful Names

Use clear, descriptive names: `student_utils.py`, not `su.py` or `module1.py`



Single Responsibility

Each module should do ONE thing well. Don't mix database + calculations + UI in one file



Keep Modules Small

If a module exceeds 200-300 lines, consider splitting it into a package



Document Everything

Add docstrings to every module, class, and function. Future you will thank you!



Use Packages for Scale

When your project grows, organize related modules into packages immediately



Follow PEP 8

Module names: lowercase with underscores. Class names: CamelCase. Constants: UPPER_CASE



Avoid Circular Imports

If A imports B and B imports A, you have a design problem — restructure your code



Avoid `import *`

Explicit is better than implicit. Always name what you import for readable code

Real Project Structure — AI/ML Project Example

AI_Project/ (Package Structure)

```
AI_Project/
├── main.py
├── requirements.txt
├── data/
│   ├── __init__.py
│   └── loader.py
├── models/
│   ├── __init__.py
│   └── neural_net.py
├── preprocessing/
│   └── transforms.py
├── utils/
│   └── metrics.py
└── training/
    └── trainer.py
```

data/

Data loading, parsing CSV/JSON files, dataset handling

models/

Model architecture definitions and neural network code

preprocessing/

Data cleaning, normalization, augmentation transforms

utils/

Metrics, logging utilities, helper functions

training/

Training loops, hyperparameter tuning, checkpoints

Interview Questions — Be Prepared!

Q1 What is a module in Python?

A: A module is a .py file containing Python code (functions, classes, variables) that can be imported and reused in other programs.

Q2 What is a package? How is it different from a module?

A: A package is a directory with `__init__.py` containing multiple related modules. A module is a single .py file; a package is a directory.

Q3 What is the purpose of `__init__.py`?

A: It marks a directory as a Python package, runs initialization code, and can control what is exported via `__all__`.

Q4 How does Python find a module when you import it?

A: Python searches: 1) Current directory, 2) PYTHONPATH, 3) Standard library, 4) Site-packages (pip installed).

Q5 What is the difference between `import X` and `from X import Y`?

A: 'import X' loads module, accessed via `X.function`. 'from X import Y' imports specific item directly, accessed as `Y`.

Q6 What is pip and when do you use it?

A: pip is Python's package manager. Use it to install third-party modules: `pip install package_name`.

Q7: What is `ModuleNotFoundError`? → Raised when Python cannot find the requested module. Fix: pip install or check path.

Quick Quiz — Test Your Knowledge!

Q1 What file makes a directory a Python package?

- A) package.py
- B) `__init__.py`
- C) main.py
- D) setup.py

Q4 Which is NOT a built-in Python module?

- A) math
- B) os
- C) pandas
- D) json

Q2 Which command installs third-party Python packages?

- A) python install X
- B) import X
- C) pip install X
- D) get-package X

Q5 Where does Python look FIRST when searching for a module?

- A) pip packages
- B) Standard library
- C) Current directory
- D) PYTHONPATH

Q3 What does 'import math as m' allow you to do?

- A) Create a new module
- B) Use `m.sqrt()` instead of `math.sqrt()`
- C) Install math
- D) Delete the module

Answers: 1-B | 2-C | 3-B | 4-C | 5-C

Session Summary



Modules are .py files — the building blocks of organized Python



Packages are directories of modules, marked by `__init__.py`



Built-in, User-Defined, and Third-Party are the 3 module types



4 import styles: `import X`, `from X import Y`, `from X import *`, `import X as alias`



Python searches: current dir → PYTHONPATH → stdlib → site-packages



Real AI/ML projects use package structures to stay organized

Thank You! Happy Coding!

Lovnish Verma | NIELIT Ropar | IndiaAI Programme | @lovnishverma